

PCCST503 – Machine Learning

Assignment 1

Design of a Safe Semantic Planner in a Finite Cartesian State Space

This submission implements a Safe Lifelong Planning A* (LPA*) planner for the finite directed Cartesian state space specified in the assignment. The design explicitly avoids bad states, minimizes transition cost through an additive scalar objective, rewards safety and reliability, and supports incremental updates to transition availability, goals, bad states, and newly inserted transitions.

1. Problem Definition

The input consists of a finite set of states embedded in $\mathbb{R}^d$, an initial state, a goal state, a set of bad states, and directed transitions. Each transition has cost, reliability, safety, and an availability flag. The planner returns a sequence of states and transitions that reaches the goal without visiting a bad state.

- Hard constraint: a bad state is never traversable.
- Primary optimization: minimize cumulative transition cost.
- Safety optimization: prefer paths whose visited states stay farther from the nearest bad state.
- Reliability optimization: prefer transitions with higher reliability.
- Dynamic operation: update the existing search state instead of rebuilding the complete graph.

2. State and Transition Representation

State: {uint64_t id; vector<double> embedding}. The embedding stores the Cartesian coordinates.

Transition: {id, from, to, cost, safety, reliability, available}. PlanningProblem stores the initial state, goal state, bad-state IDs, all states, and all transitions. PlanningResult stores success, state path, transition path, total cost, safety score, explored states, and planning time.

3. Data Structures

- unordered_map<id, State> for state lookup.
- unordered_map<id, Transition> for transition lookup.
- out_[u] adjacency lists for outgoing transitions.
- in_[v] reverse adjacency lists for predecessor evaluation.
- unordered_set bad_ for O(1)-average bad-state membership checks.
- g and rhs arrays/maps for LPA* one-step lookahead values.
- priority_queue for the LPA* inconsistent-state queue.
- Reverse BFS hop-distance map for an admissible lower-bound heuristic.

4. Safe Objective Function

Because LPA* requires an additive edge-value model, the implementation scalarizes the assignment's multi-objective criteria. For an available edge $e=(u,v)$:

$$w(e) = \text{cost}(e) + \lambda s / (d(v, B) + \epsilon) + \lambda s (1 - \text{safety}(e)) + \lambda r (1 - \text{reliability}(e))$$

Here $d(v,B)$ is the Euclidean distance from v to the nearest bad state. The first safety term becomes larger as a state approaches a bad state; the second term penalizes a low transition safety score; the reliability term penalizes low reliability. Bad states are still hard constraints, so no amount of optimization benefit can make a bad state legal.

For the safety-margin test, λ_s is set high enough that the planner chooses the farther path even when it has a larger raw transition cost. For pure cost tests, λ_s and λ_r are set to zero.

5. Heuristic Function

The heuristic is $h(u) = \text{Hops}(u, \text{goal}) \times c_{\min}$, where Hops is an unweighted shortest-path distance computed by reverse BFS and $c_{\min}$ is the smallest non-negative base transition cost among currently available edges. Since every traversable edge has base cost at least $c_{\min}$, any path with H hops has base cost at least $H \times c_{\min}$. Therefore the heuristic is a lower bound on the scalar objective whenever the additional safety/reliability penalties are non-negative. If no reverse path exists or $c_{\min}$ is zero, the implementation safely falls back to $h=0$.

6. LPA* Algorithm

The planner initializes $g(u) = \infty$ and $rhs(u) = \infty$, except $rhs(\text{start}) = 0$. Inconsistent vertices are inserted into the priority queue using the standard two-component LPA* key. `ComputeShortestPath` repeatedly processes the smallest key until the goal is locally consistent and no smaller key remains.

- If $g(u) > rhs(u)$, set $g(u) = rhs(u)$ and update successors.
- Otherwise set $g(u) = \infty$, update u , and propagate changes to successors.
- $rhs(v) = \min$ over predecessors u of $[g(u) + w(u,v)]$.
- Path reconstruction follows predecessor edges that attain the minimum $g(u) + w(u,v)$.
- Stale priority-queue entries are discarded by comparing their stored key with the current key.

7. Safety Computation

For each visited state v , the nearest bad-state distance is computed as $\min_b ||v-b||_2$. The `PlanningResult` `safetyScore` is the minimum such distance along the returned path. If there are no bad states, the value is reported as infinity. This directly corresponds to the assignment's minimum safety-distance objective.

8. Dynamic Replanning

- Transition availability change: update the affected destination vertex and rerun the incremental shortest-path procedure.
- Transition insertion: add the edge to the existing adjacency structures, update its destination, and continue incrementally.
- Goal change: change the termination target and recompute the lightweight reverse-BFS heuristic; existing g/rhs values are retained.
- Bad-state update: replace the bad-state set, refresh the affected edge evaluations and heuristic, and propagate inconsistency.
- No complete graph reconstruction is required for these updates; the persistent search structures are reused.

The supplied implementation prioritizes clarity and correctness for the finite assignment setting. For very large graphs with frequent bad-state-set changes, an optimized implementation would maintain an explicit affected-vertex index rather than touching all edge endpoints.

9. Complexity Analysis

Let $|V|$ be the number of states and $|E|$ the number of transitions. With adjacency lists, the incremental shortest-path computation has the usual LPA* bound $O((|V|+|E|) \log |V|)$ for a full replanning episode under standard assumptions. Reverse BFS for the hop heuristic costs $O(|V|+|E|)$. Computing the nearest bad state directly costs $O(|B|)$ per queried state, so a complete path's safety evaluation is $O(|P||B|d)$, where d is embedding dimension. Memory is $O(|V|+|E|+|B|)$.

The exact runtime is implementation- and update-dependent; incremental updates can be much cheaper than rebuilding and rerunning a planner from scratch because only inconsistent portions of the search state need propagation.

10. Test Cases and Experimental Results

The six assignment scenarios were implemented and executed with the supplied C++17 program. The following results are from the compiled program in this submission.

Case	Success	State path	Cost	Min bad distance	Explored
Test 1	Yes	1 -> 2 -> 3 -> 4	3.0000	∞	4
Test 2	Yes	1 -> 4 -> 5 -> 6	3.1000	1.0	5
Test 3	Yes	1 -> 5 -> 6	2.8000	1.4142	4
Test 4a	Yes	1 -> 2 -> 4	2.0000	∞	3
Test 4b	Yes	1 -> 3 -> 4	2.4000	∞	6
Test 5a	Yes	1 -> 5 -> 3 -> 4	2.0000	∞	5
Test 5b	Yes	1 -> 5	0.5000	∞	5
Test 6a	No	No path	0.0000	—	0
Test 6b	Yes	1 -> 2 -> 4	1.2000	∞	4

11. Interpretation of Tests

- Test 1: returns the unique path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.
- Test 2: state 3 is bad, so the planner rejects $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$ and selects $1 \rightarrow 4 \rightarrow 5 \rightarrow 6$.
- Test 3: the planner selects $1 \rightarrow 5 \rightarrow 6$ because its minimum bad-state distance is 1.4142, trading a higher raw cost for a larger safety margin.
- Test 4: when transition $2 \rightarrow 4$ is disabled, the existing planner replans to $1 \rightarrow 3 \rightarrow 4$.
- Test 5: changing the goal from 4 to 5 immediately changes the returned path to $1 \rightarrow 5$.
- Test 6: initially no route exists; after inserting shortcut transition $2 \rightarrow 4$, the planner discovers $1 \rightarrow 2 \rightarrow 4$.

12. Limitations and Possible Improvements

- The multi-objective assignment is converted to an additive scalar objective; this is a practical trade-off rather than a full Pareto-front optimizer.

- The direct nearest-bad-state calculation is $O(|B|)$ per state query; a spatial index such as a KD-tree could accelerate large bad-state sets.
- The current implementation uses a reverse BFS heuristic. A stronger admissible heuristic can be added when additional geometric assumptions on transition costs are guaranteed.
- An explicit affected-edge/vertex index could make large bad-state-set updates more selective.
- Optional extensions include multi-goal planning, time-dependent availability, parallel search, and a learning-based heuristic.

13. Conclusion

The resulting Safe LPA* planner satisfies the assignment's core requirements: goal reachability, zero bad-state visits, cost optimization, safety-distance optimization, practical execution, and dynamic replanning. The program is self-contained, uses C++17, and includes executable demonstrations for all six illustrative test cases.